

참고 : <https://spring.io/> <https://www.infllearn.com/course/lecture?courseSlug=%EC%8A%A4%ED%94%84%EB%A7%81%EB%B6%80%ED%8A%B8-%EA%B0%9C%EB%85%90%EC%A0%95%EB%A6%AC&unitId=97740&tab=curriculum>

스프링은 현재도 계속 발전 중이며, 스프링 생태계가 방대하기에 너무 레거시한 기술은 배제하고 현업에서 자주 쓰이는 것 위주로 공부하여 전체적인 개념 위주로 정리하고자 한다.

spring(스프링)이란?

자바 기반 오픈소스 프레임워크.

스프링의 주요한 특징을 요약하면 다음과 같다.

- IoC 컨테이너를 가진다.
- DI를 지원한다.
- 많은 수의 필터와 어노테이션을 가진다.
- MessageConverter를 가진다.
- BufferedReader, BufferedWriter를 쉽게 사용할 수 있다.

위 개념은 스프링에서만 존재하는 것은 아니고, 이를 지원하는 다른 프레임워크나 라이브러리들도 존재한다.

IoC (Inversion of Control - 제어의 역전)

객체의 생성 및 관리 등 제어권을 개발자가 아닌, 어플리케이션에 위임하는 디자인 패턴이다.

프레임워크 없이 개발을 하게 된다면 개발자가 객체의 생명주기를 관리하고 객체 간 관계는 코드 내에 직접 명시되어 있지만, 이러한 **제어**를 **역전**시켜 외부 컨테이너 또는 프레임워크가 제어 권한을 갖게되는 것이다. (프레임워크 자체도 제어의 역전 개념이 적용된 것이다.)

- spring이 여러 객체들을 스캔해서 heap 메모리에 띄운다. (spring이 객체들을 관리)
- 장점 : 각각의 객체가 메모리에 하나만 존재하고, 모든 class의 method에서 이 객체를 공유하여 사용할 수 있다. 프로그래밍의 편의성을 높여준다.

Q. spring이 객체를 메모리에 올린 것인데, 어떻게 주소를 찾아가나? A. 다음에 나올 의존성주입(DI)을 통해 가능하게 된다! 그렇다면 의존성 주입이 무엇인지 알아보자.

DI (Dependency injection - 의존성 주입) 이란?

객체가 자신의 의존성을 직접 관리하지 않고, 외부에서 객체 간의 관계를 정해주는 디자인 패턴.

- 객체지향프로그래밍의 핵심 원칙 중 하나이다.
- IoC의 한 형태이다.
- 런타임 시에 동적으로 주입한다.

의존성 한 객체 내에서 다른 객체를 사용할 때 의존성이 있다고 한다.

의존성 주입 방법

1. Constructor Injection

- 객체의 생성자를 통해 의존 객체를 주입받는다.
- 생성자가 1개만 있을 경우 @Autowired를 생략해도 의존성 주입이 된다.

```
@Service
public class UserService {
    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

이 때, UserService가 UserRepository를 의존한다고 표현한다. (또는, UserService가 UserRepository에 의존성이 있다.)

클래스 다이어그램을 그릴 때는 UserService--->UserRepository와 같이 점선으로 표현한다.

2.setter 주입

- 멤버변수값을 변경하는 setter 메서드를 통해 의존 객체를 주입받는다.

```
@Service
public class UserService {
    private UserRepository userRepository;

    @Autowired
    public void setUserRepository(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // ...
}
```

위 두 방법 외에도 다른 방법이 있지만, 주로 생성자 주입을 사용하는 것이 권장된다.

Q.그러나 Constructor injection은 생성자 호출 시에 한 번만 호출되는데, 주입받은 객체가 변한다면? A.주입받은 객체가 변경될 가능성이 있는 경우에 setter injection을 사용하면 되지만, 이 경우는 매우 드물다. 따라서 생성자 주입을 적극 사용하자.

Q.그렇다면 의존성주입을 왜 사용하고, 이것이 가지는 장점은 뭘까? A.

필터

클라이언트의 요청과 응답을 가로채어 사전에 걸러내는 역할. 인코딩, CORS와 같은 보안

- 자바에서 제공하는 Filter 인터페이스가 있고, 자바 서블릿 API에 정의되어 있다.

```
@Component
public class FilterTest implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
    }

    //실제 필터링 작업이 이루어지는 메소드

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        //이곳에 filter로직 작성
        chain.doFilter(request, response);
    }

    //필터가 제거될 때(WAS 종료 시) 호출됨
    @Override
    public void destroy() {
    }
}
```

- 개발자가 위 인터페이스를 구현하면, 웹 컨테이너에 의해 로드되고 관리된다.
- DispatcherServlet 앞단에 존재한다.
- 스프링에는 자체의 필터 기능이 있고, 필터를 직접 생성한다.

인터셉터

클라이언트의 요청이 핸들러(Controller)에 도달하기 전에 또는 핸들러가 응답을 반환하기 전에 요청 또는 응답을 가로채고 처리한다.

```
package org.springframework.web.servlet;

import javax.servlet.http.HttpServletRequest;
```

```
import javax.servlet.http.HttpServletResponse;

public interface HandlerInterceptor {
    // 요청 처리 과정에서 실행할 컨트롤러의 인스턴스가 전달됨.
    default boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler)
        throws Exception {
        return true;
    }

    // 핸들러가 실행된 후, 뷰가 렌더링되기 전 호출
    default void postHandle(HttpServletRequest request, HttpServletResponse
response, Object handler,
        @Nullable ModelAndView modelAndView) throws Exception {
    }

    // 요청 처리 완료 후 호출
    default void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler,
        @Nullable Exception ex) throws Exception {
    }
}
```

어노테이션

컴파일러가 체크하는 주석.

- 스프링에서는 어노테이션을 통해 주로 객체를 생성한다.(@Component를 보면 해당 클래스를 메모리에 로딩)

@Autowired 메모리에 있는 객체를 해당 변수에 (없다면 null 할당)

리플렉션

런타임에 클래스, 메서드, 필드, 어노테이션 등을 검사하여 수정하는 것이다.

컴파일 시에, 클래스, 메서드, 필드 등의 이름을 몰라도 인스턴스를 생성하고, 메서드를 호출하고, 필드에 접근할 수 있다.

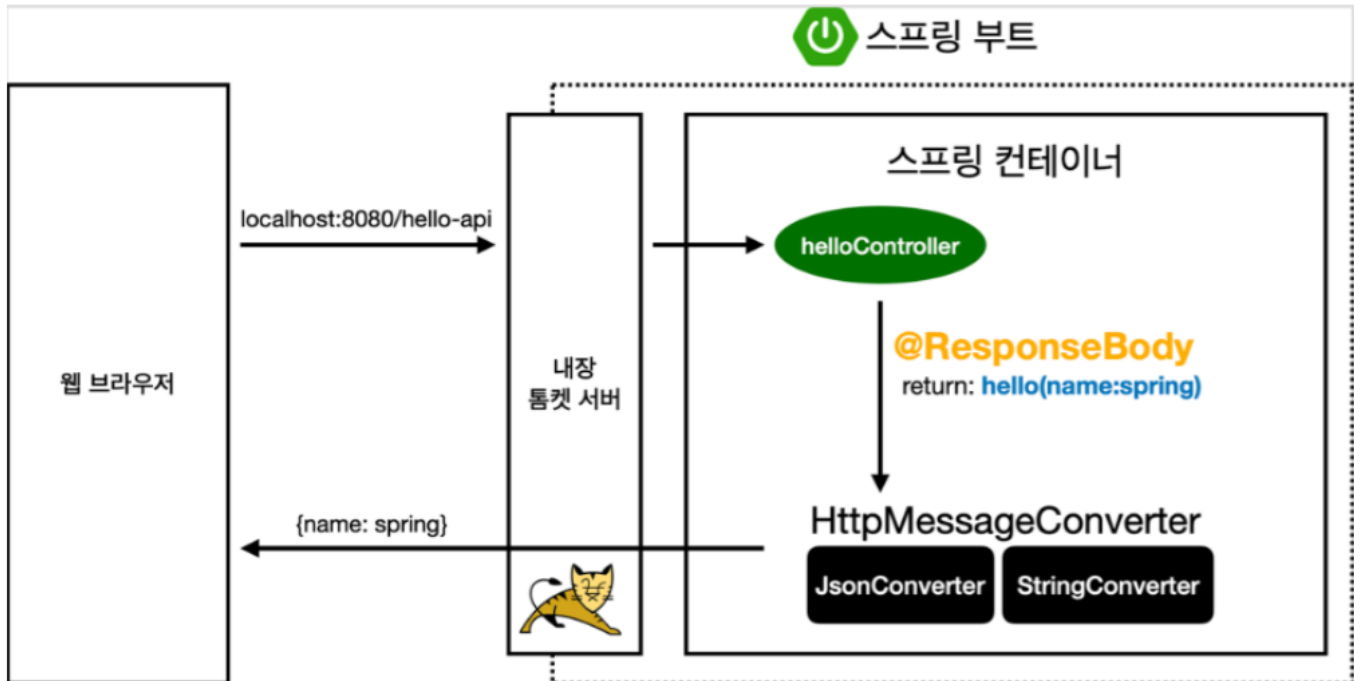
HTTP 메시지 컨버터(Message Converter)

html을 view로 띄우는 것이 아니라, json형식의 데이터를 http 메시지 바디에서 직접 읽거나 쓰기 위해 사용한다.

스프링에서는 다음의 경우에 HTTP 메시지 컨버터를 적용한다.

HTTP 요청: @RequestBody , HttpEntity(RequestEntity)

HTTP 응답: @ResponseBody , HttpEntity(ResponseEntity)



스프링부트 기본 메시지컨버터

1. ByteArrayHttpMessageConverter
2. StringHttpMessageConverter
3. MappingJackson2HttpMessageConverter

JPA (Java Persistence API)

데이터를 생성한 프로그램의 실행이 종료되더라도 사라지지 않는 데이터 특성

- 자바 프로그래밍을 할 때 영구적으로 데이터를 저장하기 위한 API이다.
- ORM기술이다.
- 반복적인 CRUD 작업을 생략하게 해준다. (함수를 통해)
- 영속성 컨텍스트를 가지고 있다.
- DB와 OOP의 불일치성을 해결하기 위한 방법론을 제공한다.(DB는 객체 저장이 불가능)
- OOP의 관점에서 모델링을 할 수 있게 해준다.(상속,컴포지션,연관관계)
- 방언 처리가 용이하여 Migration하기 좋음. 유지보수에도 좋음.

ORM (Object Relational Mapping)

자바 객체를 통해 관계형 데이터베이스의 데이터를 자동으로 생성해주는 것.

1. 자바 class를 생성한다.
2. 자바 class를 통해 데이터베이스 테이블을 자동생성한다. (JPA의 인터페이스 규칙을 통해) input : DML(Delete, update, insert) output : select

자바 connection 요청 DB 세션 오픈 자바 쿼리 전송 data 응답 - DB와 자바의 데이터 타입이 다름

영속성 컨텍스트

자바가 저장해야할 모든 메타데이터들을 영구적으로 저장하는 논리적인 개념이다.(컨텍스트는 대상의 모든 정보를 말한다.)

자바 프로그램과 데이터베이스 사이에서 데이터를 관리한다. 데이터베이스와 영속성 컨텍스트의 데이터는 동기화된다.

Q.왜 바로 DB에 접근하지 않고 영속성 컨텍스트를 한 번 거치는 것일까? A.영속성 컨텍스트는 캐시의 용도라고 볼 수 있다. 데이터베이스 접근

Spring Boot

Spring을 보다 빠르고 쉽게 사용할 수 있도록 만든 도구.

Spring과의 차이?

- 내장 서버(기본:톰캣)가 존재한다.
- 스프링 MVC, Data JPA, Security 등의 기능을 자동으로 설정 (별도 설정 파일 X)
- 컨테이너, 빈 객체 등록, 의존성 설정을 자동화
- 그 자체로 실행 가능한 JAR파일 생성 가능
- Actuator이라는 상태 모니터링과 관리를 위한 기능을 제공

- spring은 .html, .css, .png와 같은 정적 파일을 요청할 수 없다! - 스프링은 URL요청을 막아놓음. 요청 시 무조건 자바를 거쳐야.

spring boot 프로젝트 실습

Project

☒ Gradle - Groovy
 ☐ Gradle - Kotlin
 ☐ Maven

Language

☒ Java
 ☐ Kotlin
 ☐ Groovy

Spring Boot

☐ 3.3.0 (SNAPSHOT)
 ☐ 3.3.0 (M3)
 ☐ 3.2.5 (SNAPSHOT)
 ☐ 3.2.4
 ☐ 3.1.11 (SNAPSHOT)
 ☒ 3.1.10

Project Metadata

Group

myproject

Artifact

HCI

Name

HCI

Description

Demo project for Spring Boot

Package name

myproject.HCI

Packaging

☒ Jar
 ☐ War

Java

☐ 22
 ☐ 21
 ☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Thymeleaf

TEMPLATE ENGINES

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GENERATE CTRL + ⌘

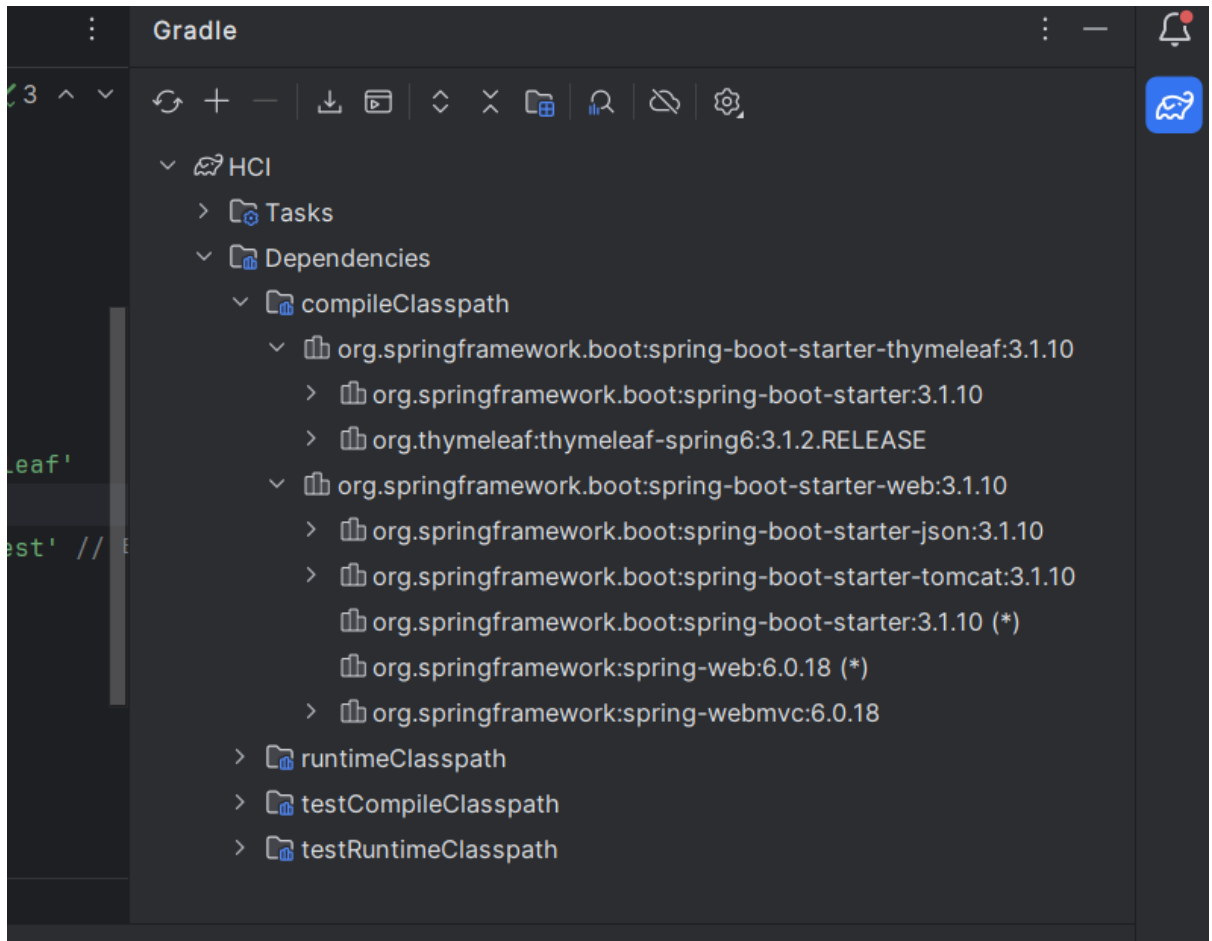
EXPLORE CTRL + SPACE

SHARE...

언어 : JAVA 17, spring boot 3.1.10 IDE : intelliJ

gradle 자바 프로젝트 빌드, 관리 도구.

- 외부 라이브러리를 가져와 빌드를 자동화하고, 의존 관계를 정리한다.
 - ex) `build.gradle` 파일의 `dependencies`에 적힌 라이브러리와 의존 관계에 있는 것들을 모두 자바 프로젝트로 가져온다. 아래 Gradle tool window에서 의존관계를 볼 수 있다.



- .gitignore 파일의 기본 템플릿을 제공한다.

spring boot 라이브러리

spring-boot-starter 공통

- spring-boot
 - spring-core
- spring-boot-starter-logging - logback, slf4j - 많이 쓰는 것을 표준으로. - System.out.println 대신 로그파일 관리 **spring-boot-starter-web**
- spring-boot-starter-tomcat
- spring-webmvc **spring-boot-starter-thymeleaf** 템플릿 엔진(View)
- html 코드가 브라우저에 잘 디스플레이 되도록 함.

spring-boot-starter-test

- junit 테스트 (최근에 5버전으로 넘어가는 추세.)
- spring-test : 스프링과 통합해서 테스트를 도와주는 라이브러리
- 그 외 mockito, assertj 등 편리하게 도와주는 도구들 포함

Spring boot 프로젝트 시작하기

1. <https://start.spring.io> 를 통해 스프링부트 프로젝트를 만들 수 있다.

The screenshot shows the Spring Boot Start web application. It is divided into several sections for configuring a new project:

- Project:**
 - ☒ Gradle - Groovy
 - ☐ Gradle - Kotlin
 - ☐ Maven
- Language:**
 - ☒ Java
 - ☐ Kotlin
 - ☐ Groovy
- Spring Boot:**
 - ☐ 3.3.0 (SNAPSHOT)
 - ☐ 3.3.0 (M3)
 - ☐ 3.2.5 (SNAPSHOT)
 - ☐ 3.2.4
 - ☐ 3.1.11 (SNAPSHOT)
 - ☒ 3.1.10
- Project Metadata:**
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
 - Packaging: ☒ Jar ☐ War
 - Java: ☐ 22 ☐ 21 ☒ 17
- Dependencies:**
 - Thymeleaf** (TEMPLATE ENGINES): A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

At the bottom, there are three buttons: **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and **SHARE...**

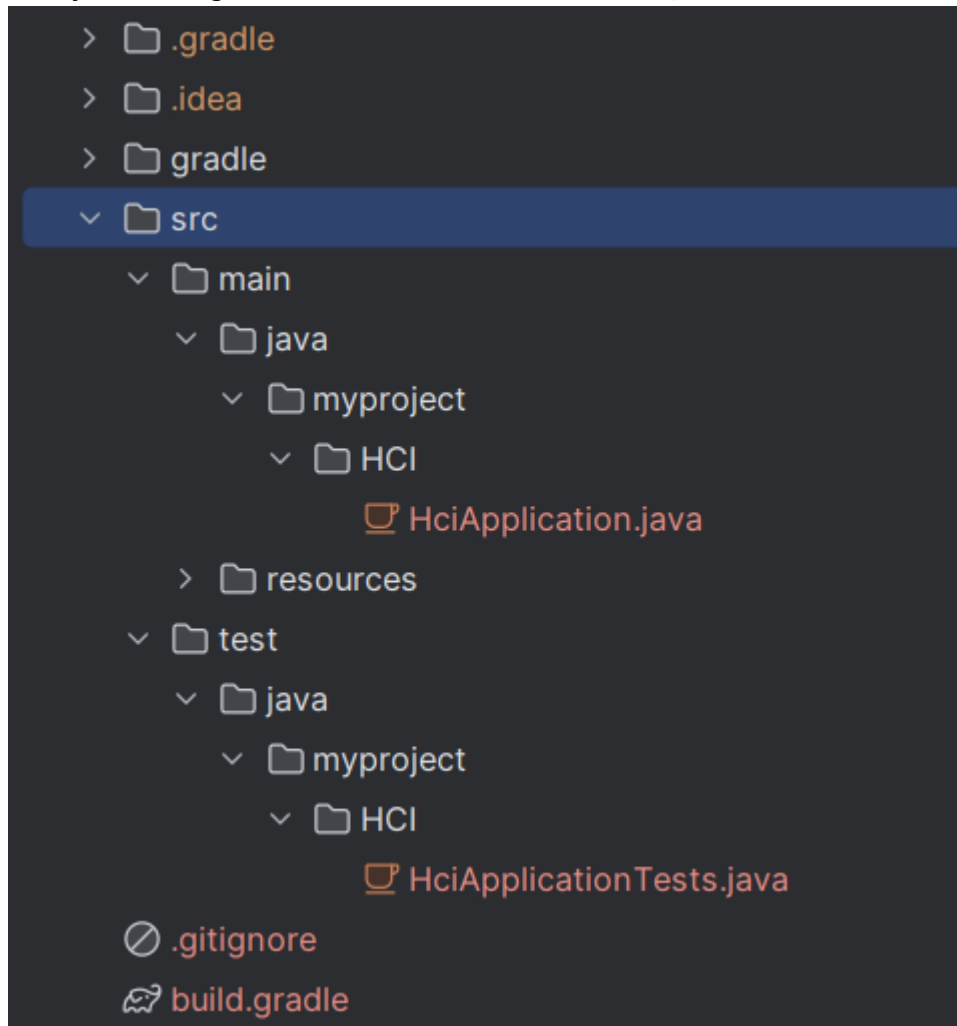
jar 독립적인 자바 프로젝트를 압축한 파일. 쉽게 실행이 가능하다.

- JVM에서 실행하므로 별도의 웹서버, WAS가 필요하지 않다.

war servlet, jsp 컨테이너에 배치할 수 있는 web application 압축파일.

- jar 파일의 일종으로, JSP, Servlet, html, javascript 등 웹 관련 자원을 포함함.
- 별도의 웹서버, WAS 필요

2. intellij에서 방금 generate한 프로젝트 파일 중 `build.gradle`을 연다.

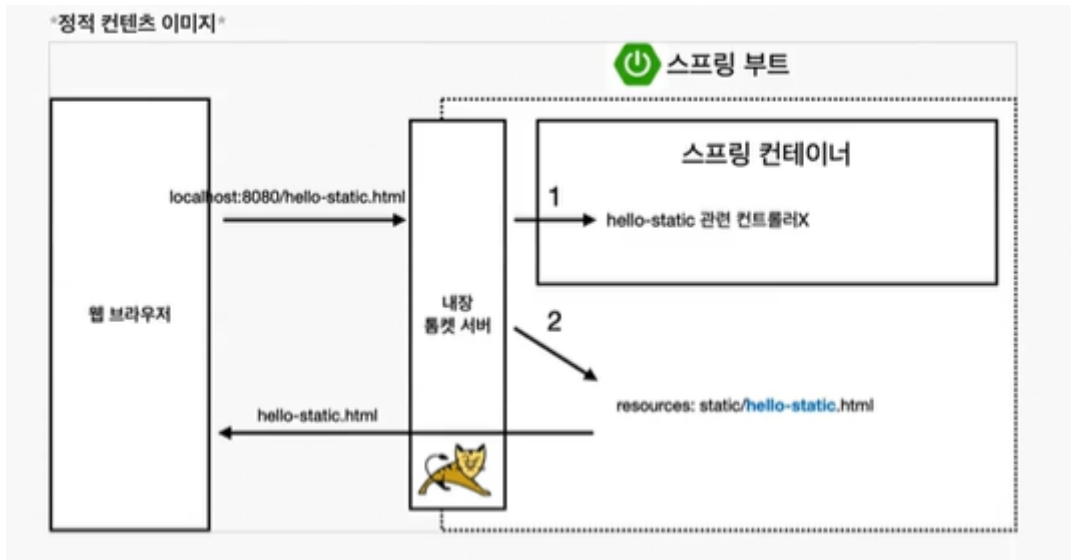


- gradle이 위와 같이 디렉토리 구조를 만든다.
- `resources`에는 자바 파일을 제외한 나머지 웹 관련 정적 파일, 설정파일이 포함된다.

3. spring boot는 가장 처음 static폴더의 index.html를 실행한다.

- 웹 어플리케이션의 첫번째 진입점은 controller !

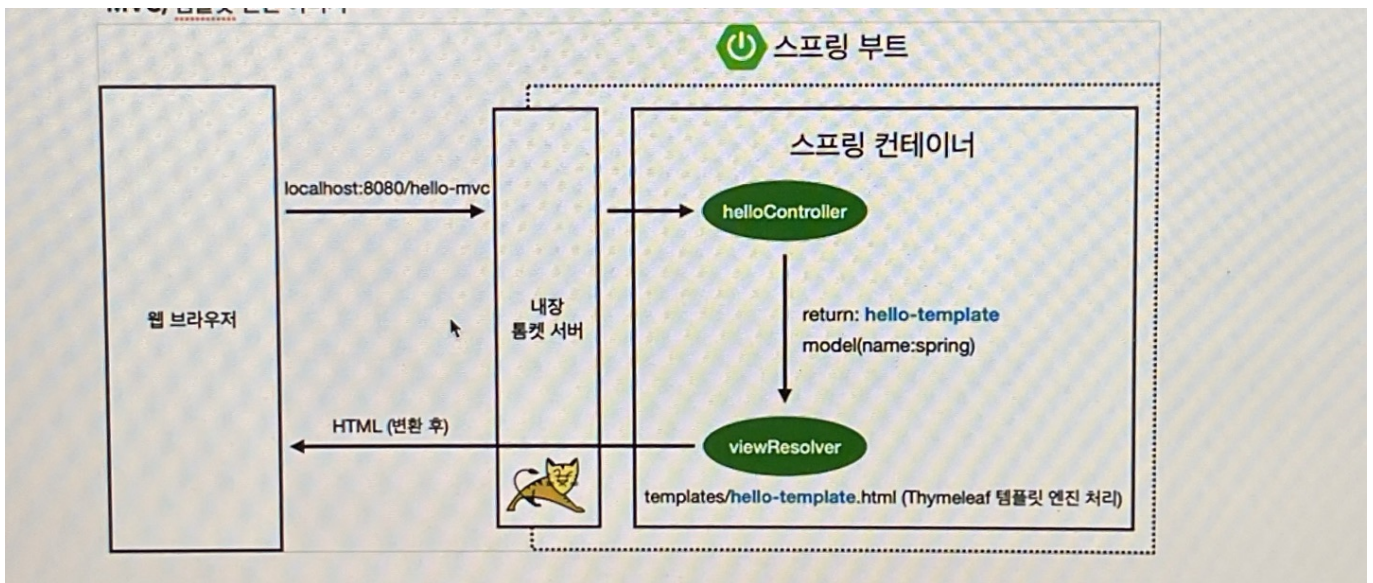
spring boot가 요청을 처리하는 방법

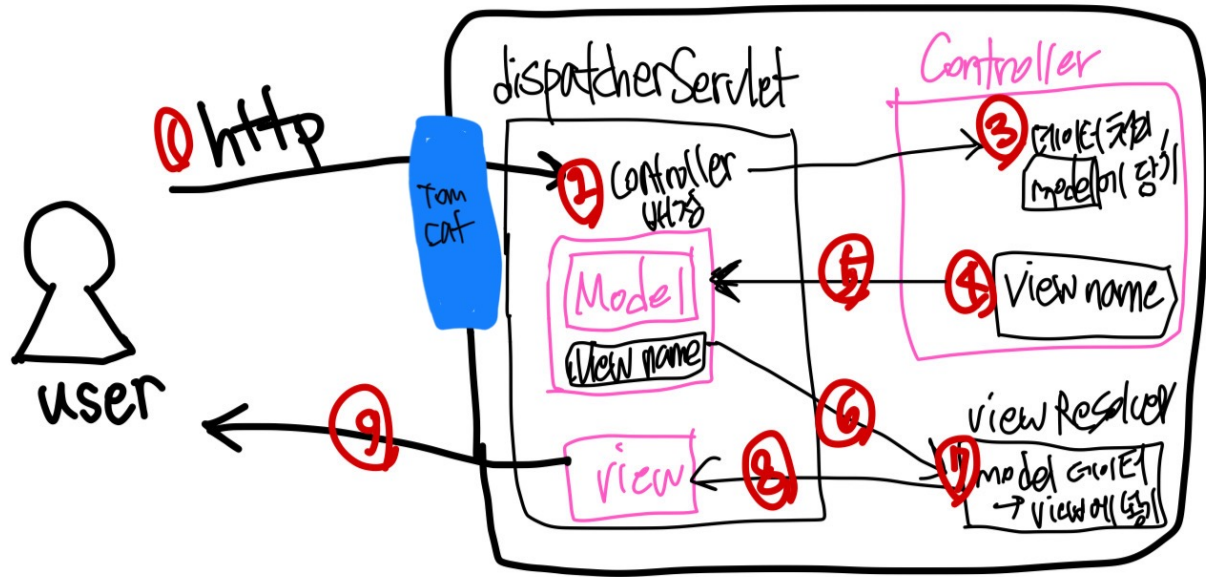


1. 정적 콘텐츠

2. MVC와 템플릿엔진 **MVC**:Model, View, Controller로 UI영역과 비즈니스 로직을 분리한 디자인 패턴.

- **Model** 자바 객체로 표현되며, 데이터를 담고 있다.
- **View** 사용자에게 보여지는 화면. Thymeleaf 등의 템플릿 엔진을 통해 생성된다.
- **Controller** 클라이언트 요청 처리 결과를 model에 저장하고, view로 전달하여 응답한다.
- Spring MVC는 **Front controller** 구조!: 클라이언트 요청을 단일 진입점으로 라우팅하고, 중앙집중식으로 처리한다.





DispatcherServlet Front controller의 역할로, 제일 먼저 http request를 처리한다. (controller의 controller)

- web.xml 또는 자바 구성
- 서블릿 컨테이너(톰캣) 등에 의해 관리됨
- 스프링 MVC의 큰 장점은 DispatcherServlet의 인터페이스만 구현하여 코드를 변경하지 않고, 원하는 기능의 컨트롤러를 만들 수 있다!

viewResolver view를 찾아주고 template를 연결시켜준다.

HelloController.java

```

@Controller
public class HelloController {
    @GetMapping("hello-mvc")
    public String helloMvc(@RequestParam("name") String name, Model model) {
        model.addAttribute("name", name);
        return "hello-template";
    }
}
  
```

- RequestParam의 기본값은 required = true이므로 localhost:8080/hello-mvc?name=spring!!! 로 parameter 값을 적어주어야 한다.

hello-template.html

```

<html xmlns:th="http://www.thymeleaf.org">
<body>
  <p th:text="'hello ' + ${name}">hello! empty</p>
</body>
</html>
  
```

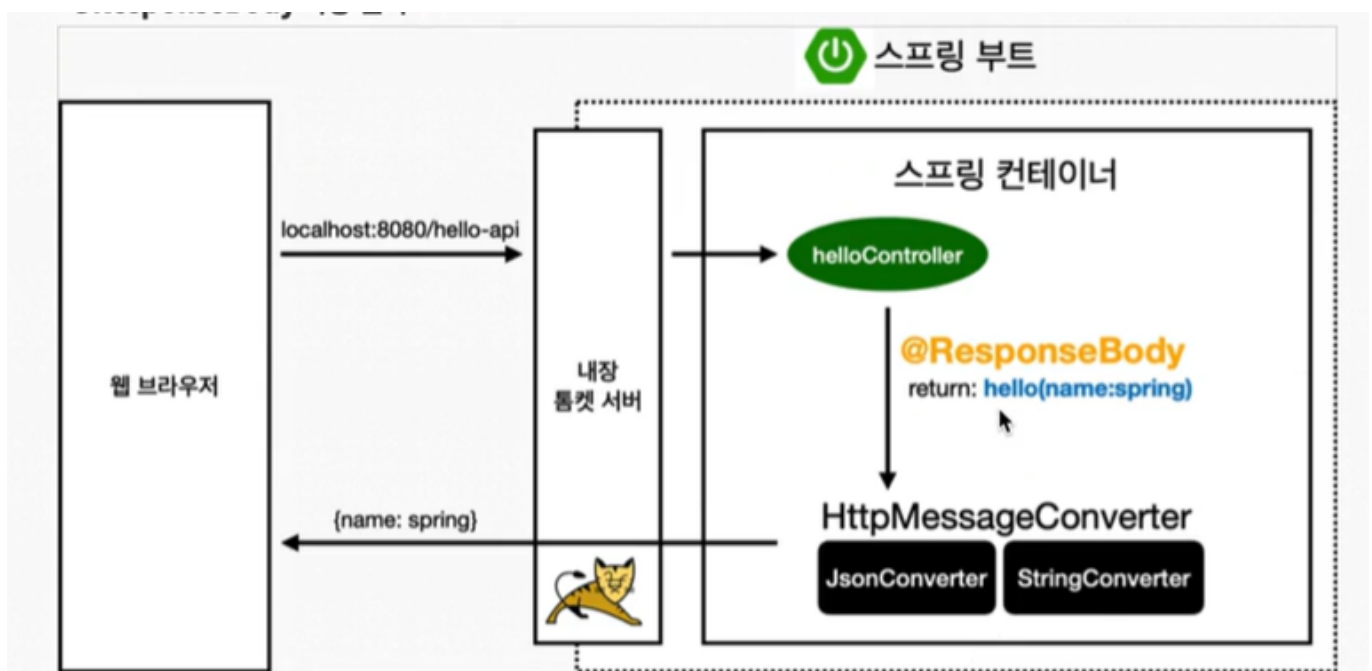
```
</body>
</html>
```

- `${name}`에 모델에서 키 값이 name인 것의 값으로 치환한다.
- `Thymeleaf`는 컨트롤러가 모델에 담은 데이터를 HTML 템플릿에 전달하여 동적인 내용을 렌더링

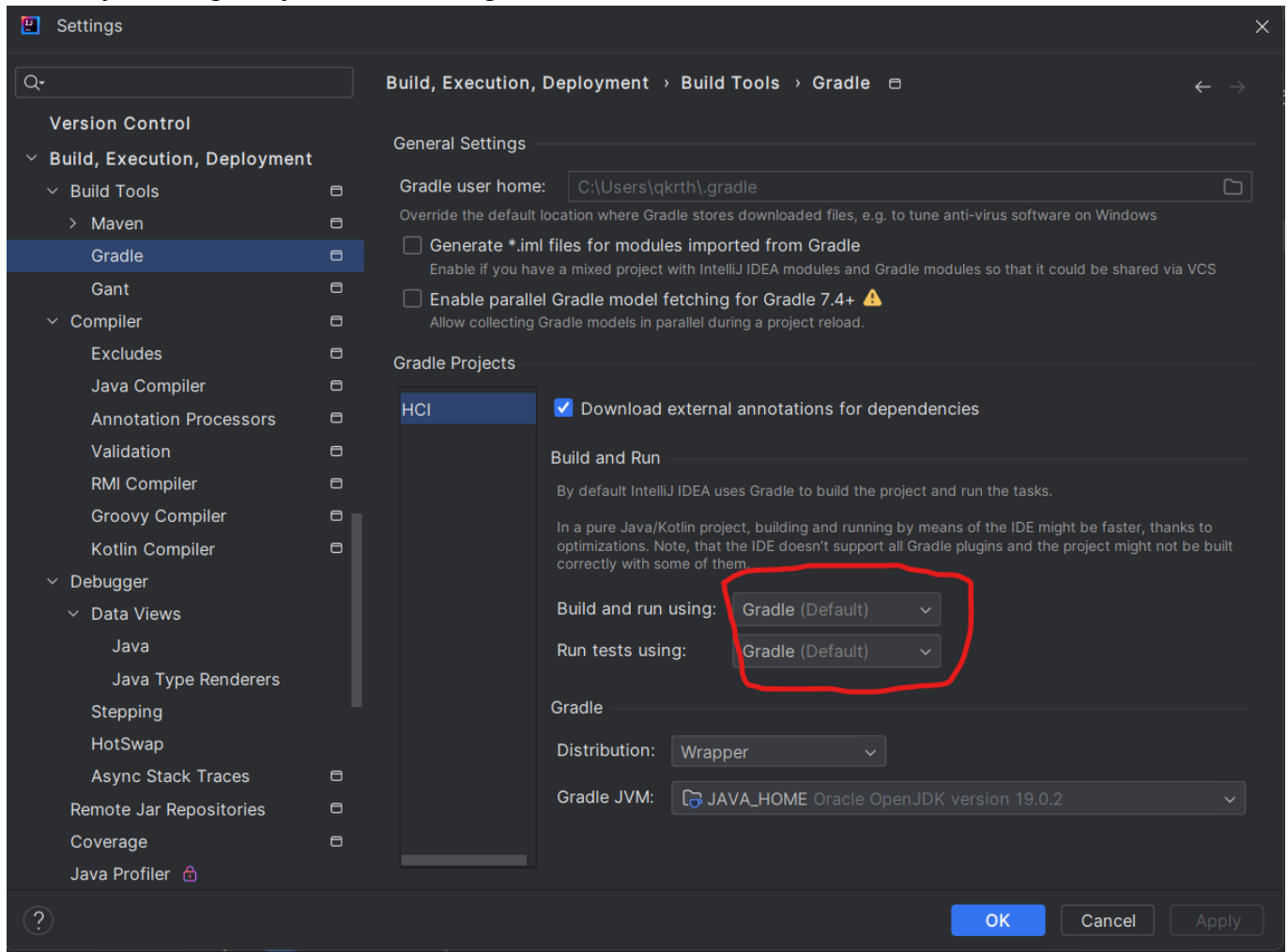
```
@GetMapping("hello-string")
@ResponseBody
public String helloString(@RequestParam("name") String name) {
    return "hello " + name; // "hello spring"
}
```

3. API

`@ResponseBody`를 통해 http의 body에 내용을 직접 실어 보낸다.



cf) intelliij의 setting에서 java를 실행할 때 gradle을 통해서 실행할 지 설정할 수 있다.



사진에서 gradle을 intelliij로 바꾸면 된다.